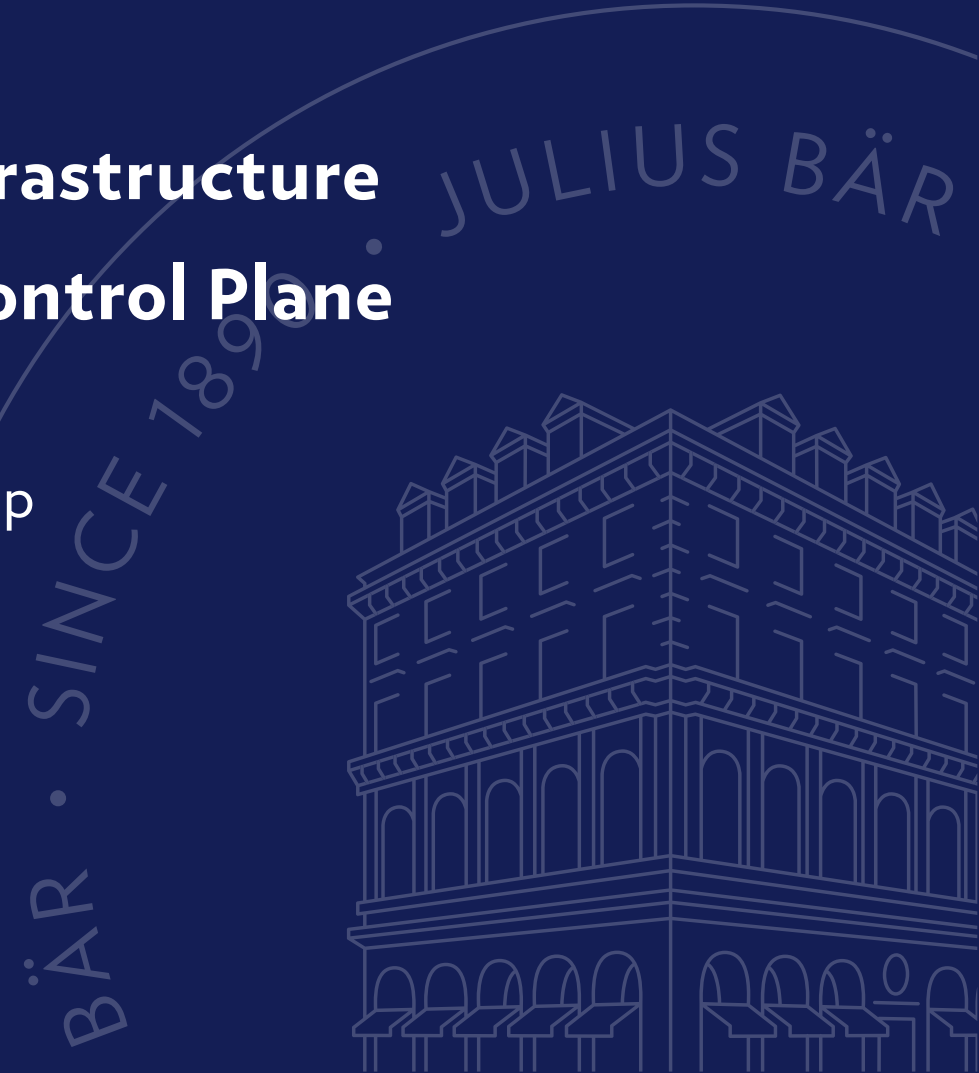


Julius Bär

Orchestrating enterprise infrastructure from a single Cloud Native Control Plane

Cloud Native Bern Meetup
May 2026

Presented by Angelo Gargiulo & Cyrill Näf





Facts & Figures

Julius Baer is a leading pure-play Swiss global wealth manager, offering comprehensive services, investment solutions, and wealth planning to private clients and institutions worldwide.

- Founded in Zurich in 1890s
- 7'400 employees in 25 countries
- IT hubs in CH, Spain, Luxembourg and Singapore
 - Classic on-prem infrastructure footprint
 - Mainly Linux & Windows VMs (VMware)
 - Tanzu Kubernetes Platform (TKGI)
- Developer platform on Azure
 - 800 Developers and Application Responsibles
- Production workloads on Azure (NEW)
 - Moving first business applications this year

Our Journey

Our story begins in 2016

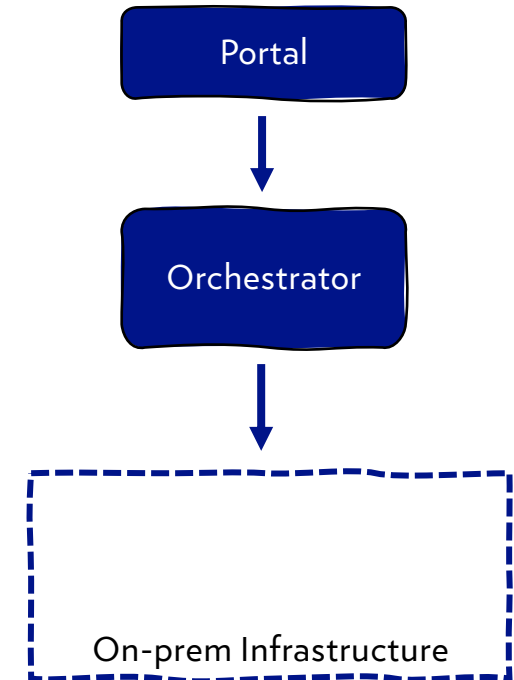
- Julius Baer was replacing its core banking system in Asia
- 18 new Dev, Test and Migration environments required
 - Linux VMs
 - Oracle databases
 - File shares & file transfers
 - MQ queues
 - Load balancing
 - Firewall rules
- All infrastructure was installed & configured manually
- Steps needed to be planned well in advance (waterfall)
- ☹️ Infra setup took **almost 1 year to complete**



Our automation journey: Chapter 1

Central orchestrator, central automation team

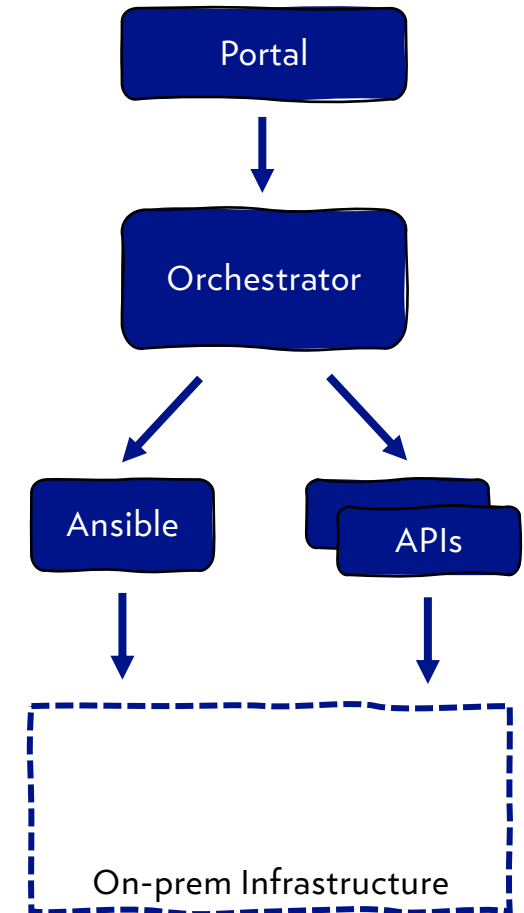
- Introduction of VMware vRealize Automation (Aria) suite (portal + workflow engine + adapters)
- Server deployment process was automated first
- Server blueprints were published in a central web-portal
- A centralized automation team was established
- Approach worked well for server deployments
→ 😊 **VM deployment within 30 minutes**
- Scaling out to other domains (databases, storage, network, security) became hard
- Central team lacked domain-specific knowledge in other areas
- Orchestrator was complex, shared development across teams was difficult
- ☹️ Central team soon became a **bottleneck**



Our automation journey: Chapter 2

Decentralized automation strategy

- We introduced an **additional abstraction layer**
- Each tech domain may automate using their own tools
- We agreed & published **automation guidelines**:
 - Central web-portal as single order entry point (validation, approvals)
 - Domains must expose their APIs to the orchestrator
 - Orchestrator orchestrates E2E processes across domains
 - Specific steps are always handled by the orchestrator (DNS, AD, IAM, CMDB)
- ☺ Bottleneck of central automation team was resolved
- ☹ Over time a **large sprawl** of “technology specific APIs” (silos)
- ☹ Some teams **lacked skills** for proper API implementation



Strategic shift to Hybrid Cloud



Strategic shift to Hybrid Cloud

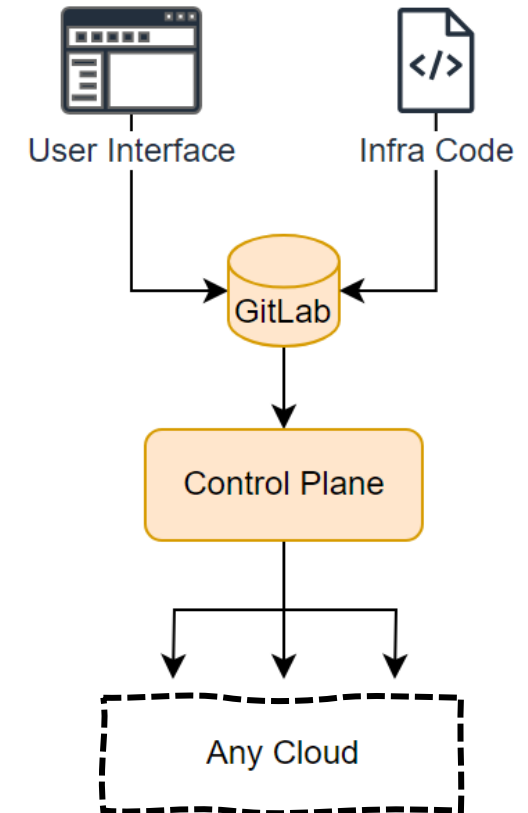
From workflow orchestration to a control plane

Pain-points

- Our on-prem tooling was not ideal for public cloud automation
- All our ordering processes were **UI based**
- Developers needed to **switch context** & tools often
- Orchestrator API was not suited for end-users
- Orchestrator workflows were **not stateful**, only day 1

New capabilities required

- Bring the public cloud experience to on-prem
- Teams can order infrastructure resources as code
- Offer a simple, abstract syntax which is easily understood (e.g. YAML)
- Store infra code alongside application code (in **Git**) → apply same compliance
- Tool that tracks **desired vs actual state** and continuously reconciles → **Control plane**



Our automation journey: Chapter 3

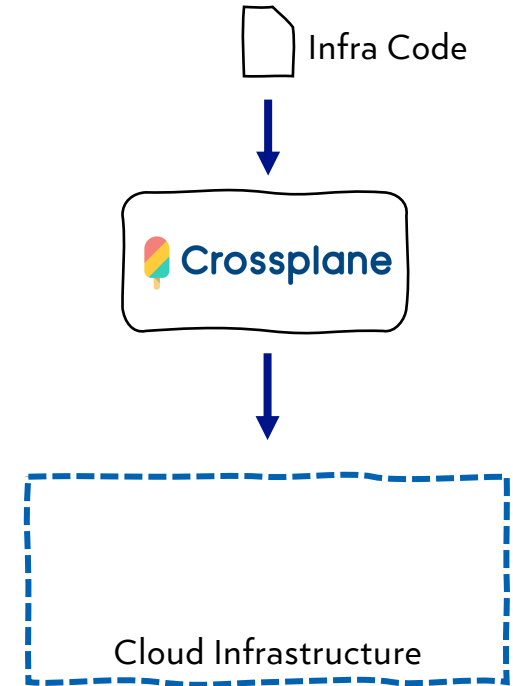
Why we chose Crossplane to build our control plane

- We evaluated various Infra as Code tools
 - Ansible
 - Azure Bicep
 - Terraform
 - Crossplane

Key requirements

- Offer a **well-known format** (inspired by the K8S API) to our users
- Cloud **provider agnostic** solution for multi-cloud capability
- **Declarative** over imperative approach
- Continuous (online) state management (unlike Terraform)
- Marketplace of existing providers maintained by a **large community**

We chose **Crossplane** because it ticked all the boxes for us



Our automation journey: Chapter 4

Decentralized control plane

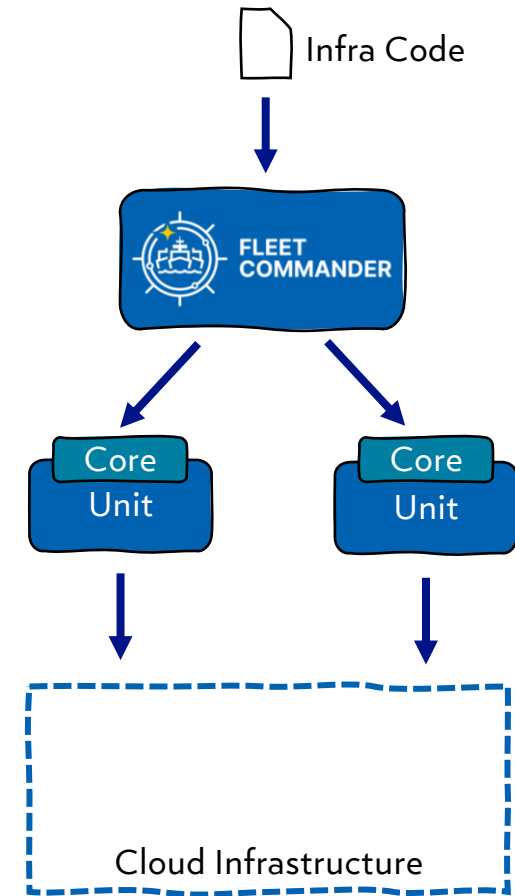
Apply our learnings to control plane architecture

- ✓ Avoid the bottleneck of a central team
- ✓ Maintain a decentralized automation approach
- ✓ Eliminate API sprawl and bespoke APIs
- ✓ Standardized integration with technology domains

Crossplane with decentralized architecture = Fleet Commander

Commander + Core → **centrally managed**

Units → **decentralized**



How?

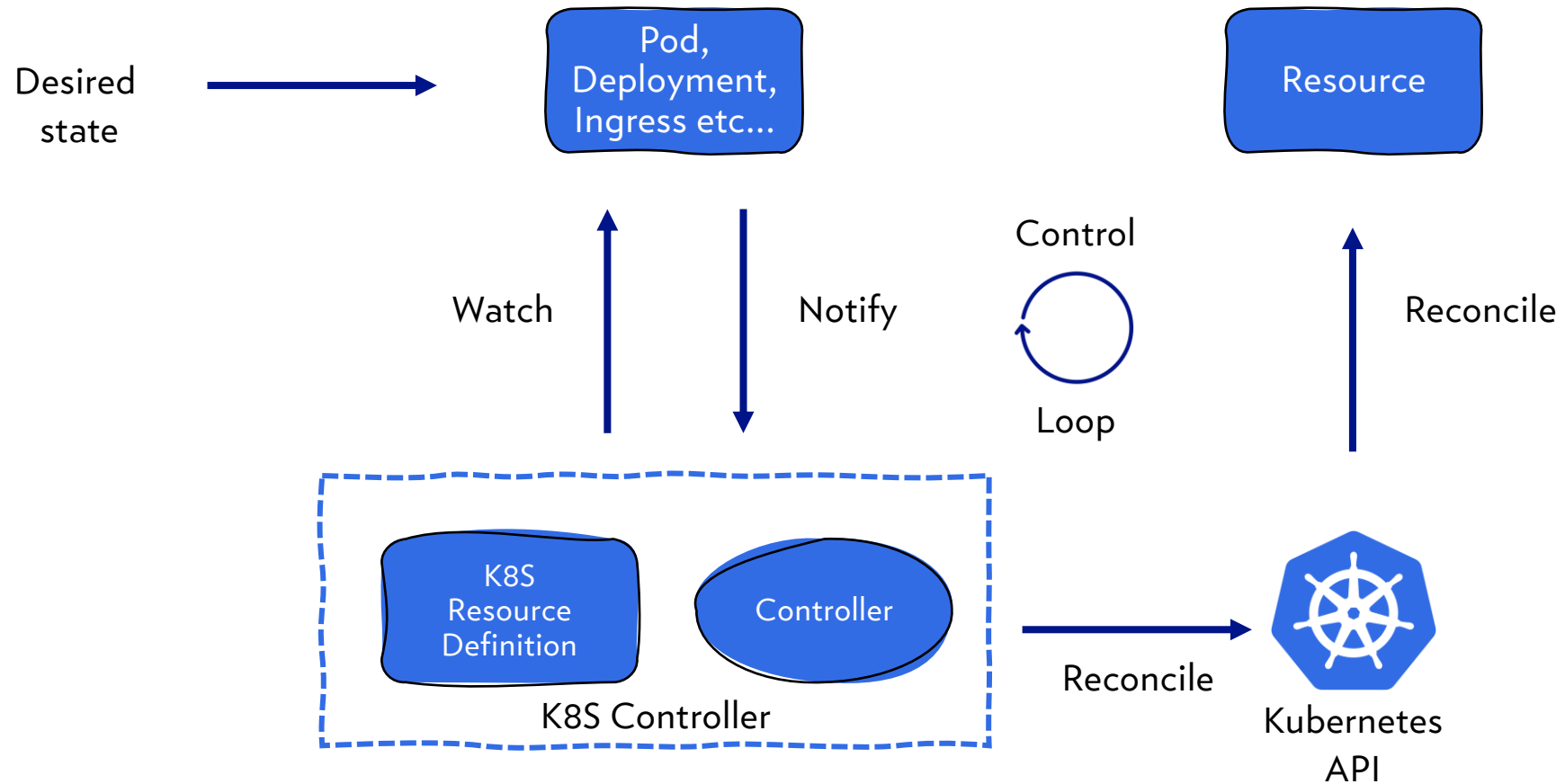


Introducing Kubernetes + Crossplane for orchestration



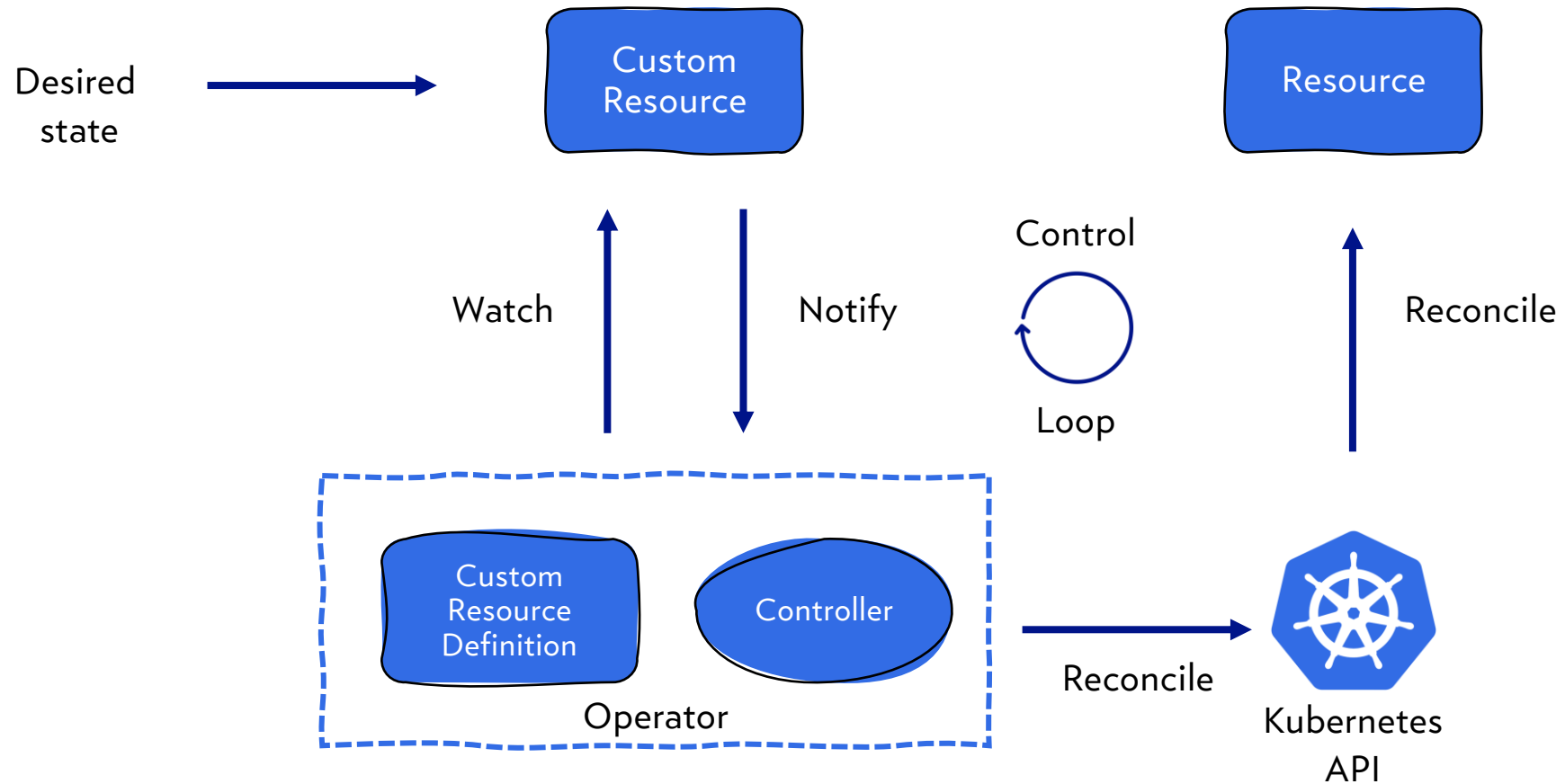
Kubernetes (default)

Control loop on K8S resources



Operator pattern

Control loop on custom resources

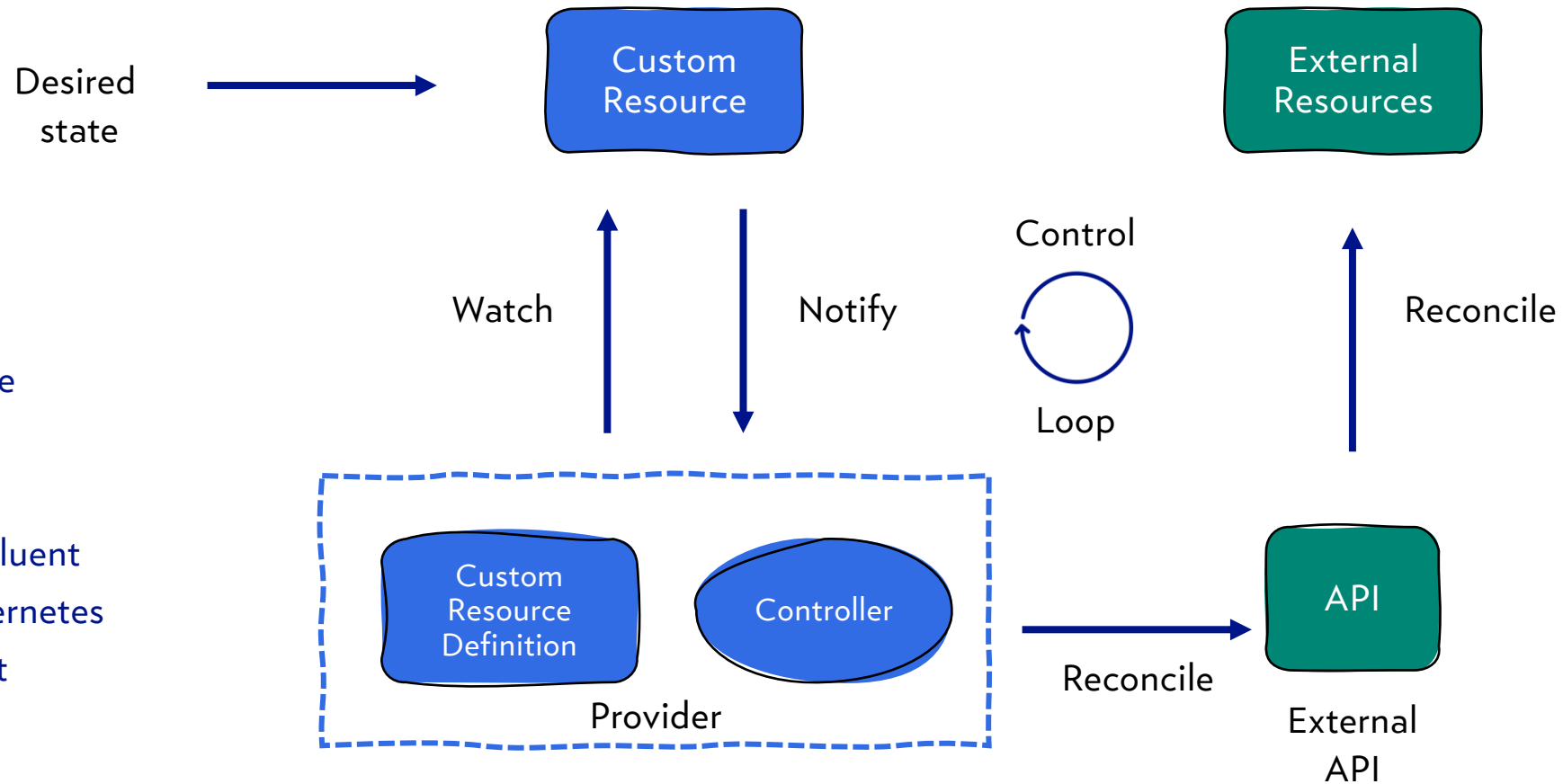


Crossplane providers

Operators for external resources

Examples

- provider-azure
- provider-aws
- provider-gcp
- provider-confluent
- provider-kubernetes
- provider-vault



Crossplane composition

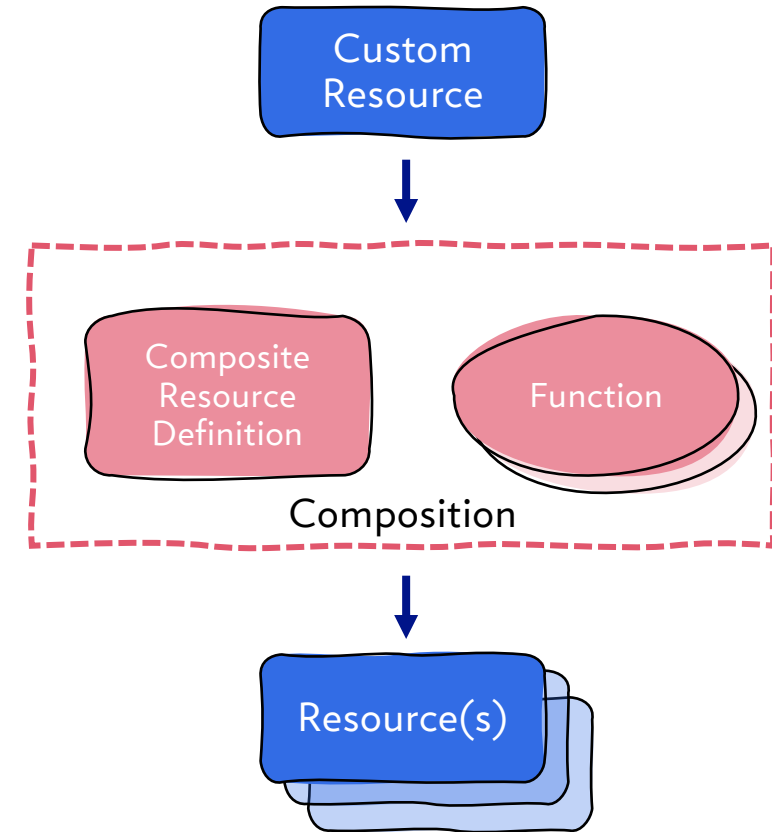
Harness the power of custom resource definitions

Composite resource definition

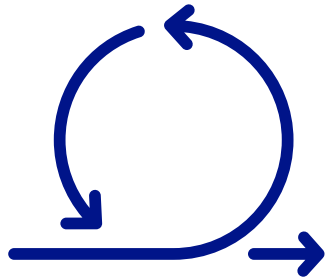
- Define the **Public API** of the Composition.
- When the Composition is deployed, Crossplane creates a standard Kubernetes **Custom Resource Definition**.

Function

- The implementation of the composition.
- Handles the CR and **decompose to one or multiple Kubernetes resources**.
- (hint: including resources defined by Crossplane providers and other compositions)



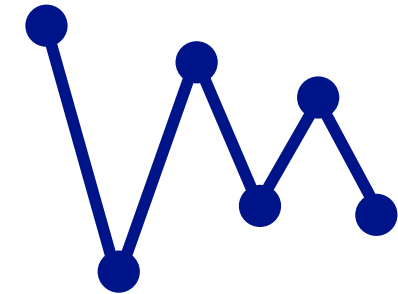
Why a stateful API as your abstraction layer?



Day 2 is the **hard** part



Your **customers** want to
declare intent, not execute
steps



One API surface **hides**
complexity – provides
flexibility

Distributed Control Planes

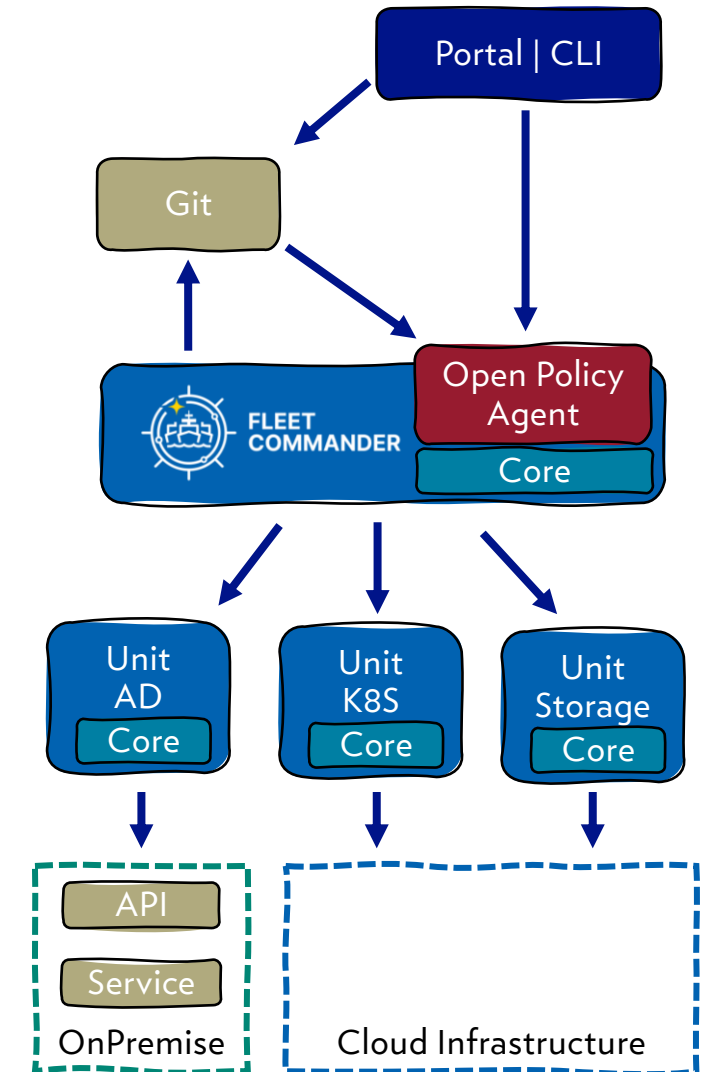
Scaling Automation Without Central Bottlenecks

Key Points:

- Central Commander exposes public API
- Central instance handles distribution
- Specialized Unit instances manage domain-specific knowledge
- Each Unit has only domain-specific providers & permissions

Benefits:

- Domain-focused technology teams
- Enhanced security
- Isolated changes help preventing cross-domain impact
- No central bottleneck team



Distributed Control Planes

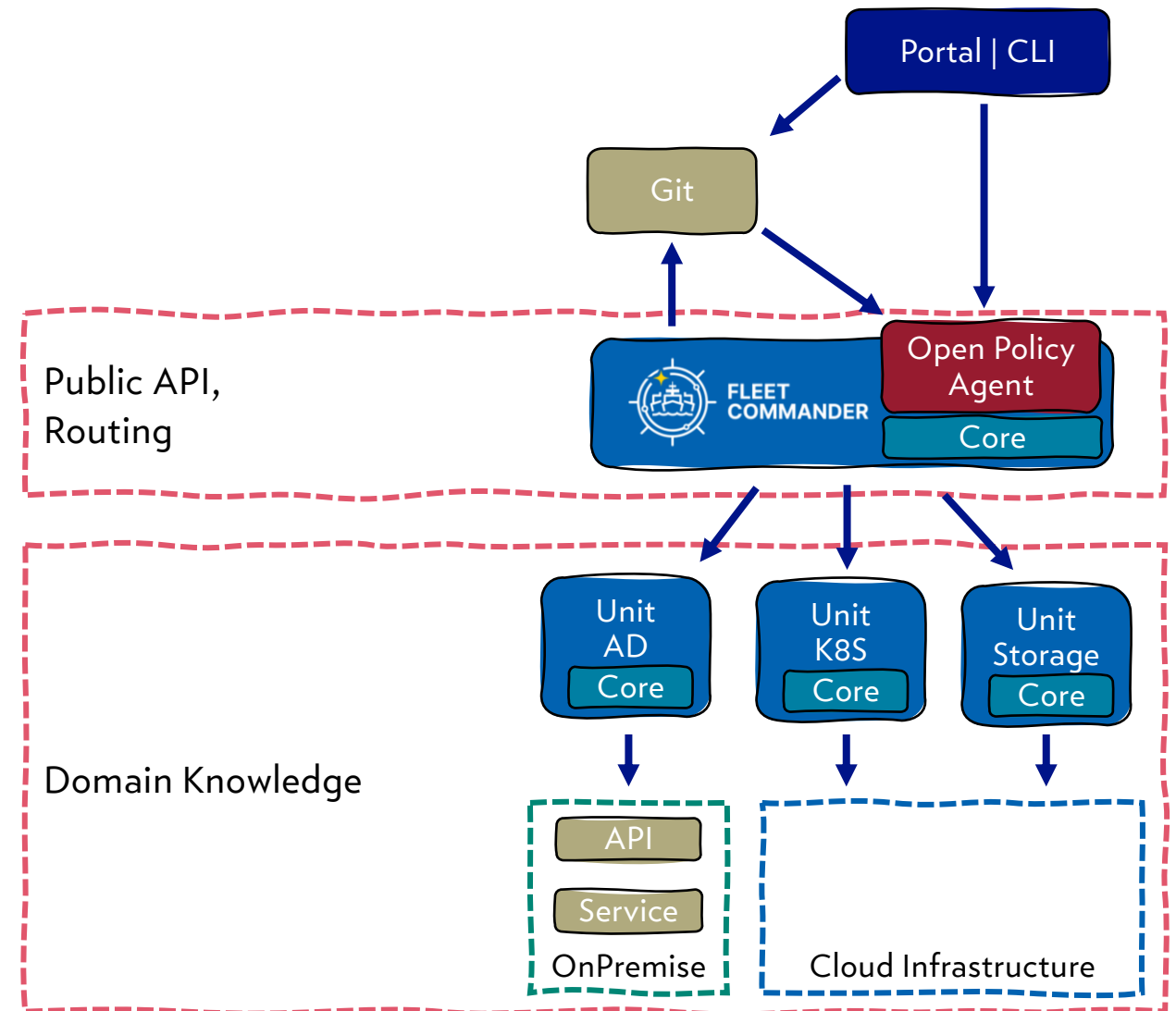
Scaling Automation Without Central Bottlenecks

Key Points:

- Central Commander exposes public API
- Central instance handles distribution
- Specialized Unit instances manage domain-specific knowledge
- Each Unit has only domain-specific providers & permissions

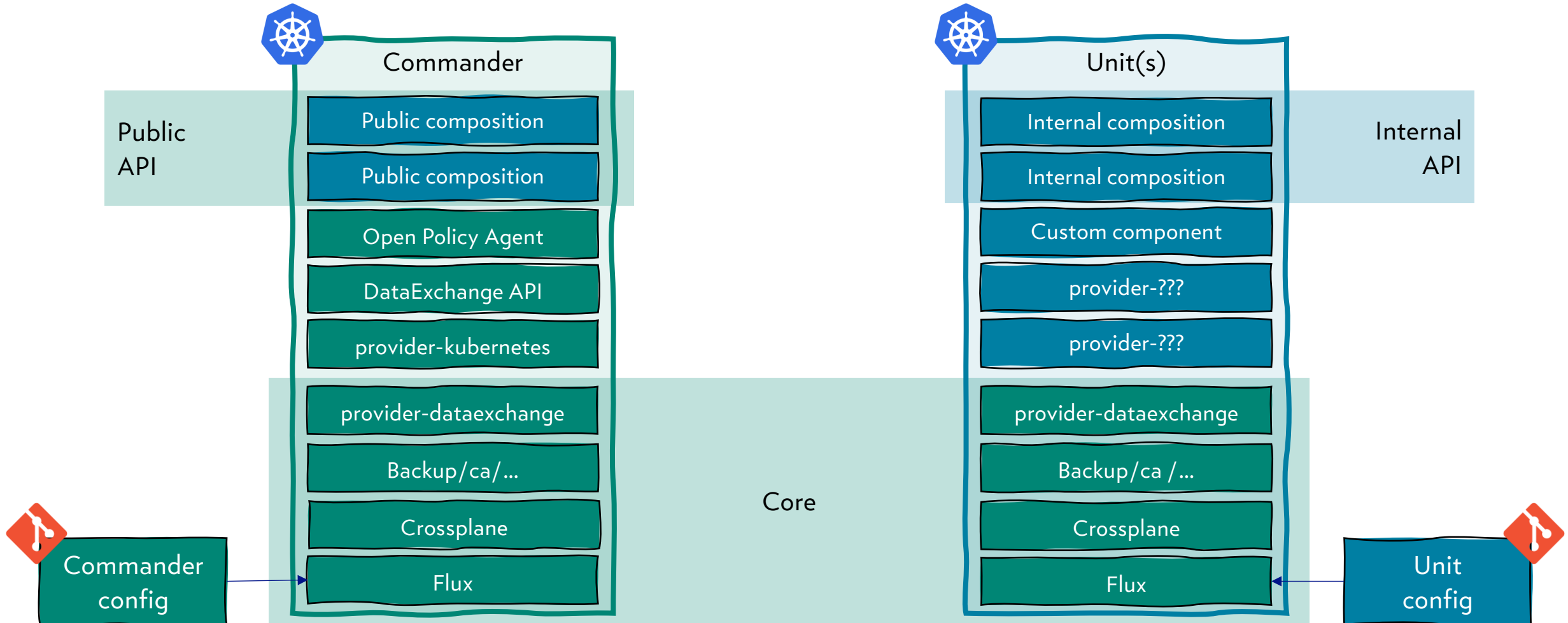
Benefits:

- Domain-focused technology teams
- Enhanced security
- Isolated changes help preventing cross-domain impact
- No central bottleneck team



Fleet commander

Shared responsibility model



How does it feel for the customer?

A Fleet Commander Resource

Simple by default, extensible for power users — the two configs you're showing tell that story visually

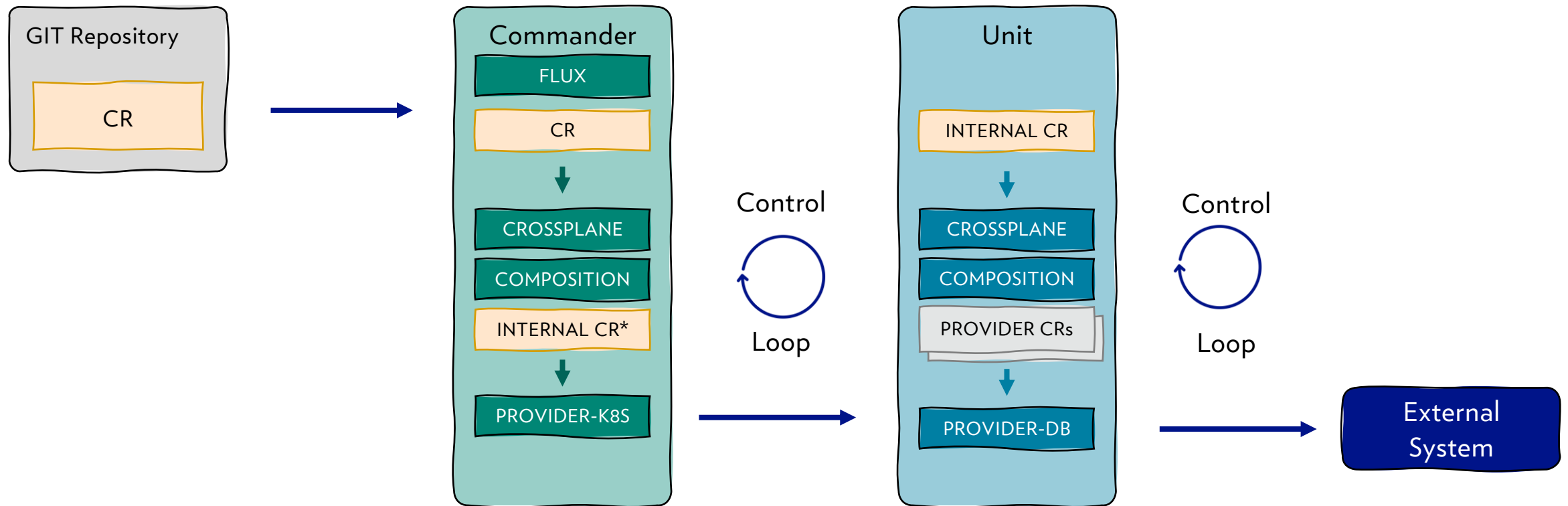
Complexity lives in the **composition**, not the **consumer**

```
apiVersion: codeak.juliusbaer.com/v1alpha1
kind: KubernetesCluster
metadata:
  name: aks-dewc-dev-tid-1
spec:
  location: germanywestcentral
  nodePools:
    - name: userpool1
```

```
apiVersion: codeak.juliusbaer.com/v1alpha1
kind: KubernetesCluster
metadata:
  name: aks-dewc-dev-tid-1
spec:
  allowDeletion: false
  enableBackup: false
  location: germanywestcentral
  maintenanceWindow:
    dayOfWeek: Friday
    startTime: "09:00"
  nodePools:
    - enableAutoScaling: false
      name: userpool1
      nodeCount: 2
      sysctlConfig:
        fsFileMax: 65536
    - enableAutoScaling: false
      name: userpool2
      nodeCount: 10
```

Example

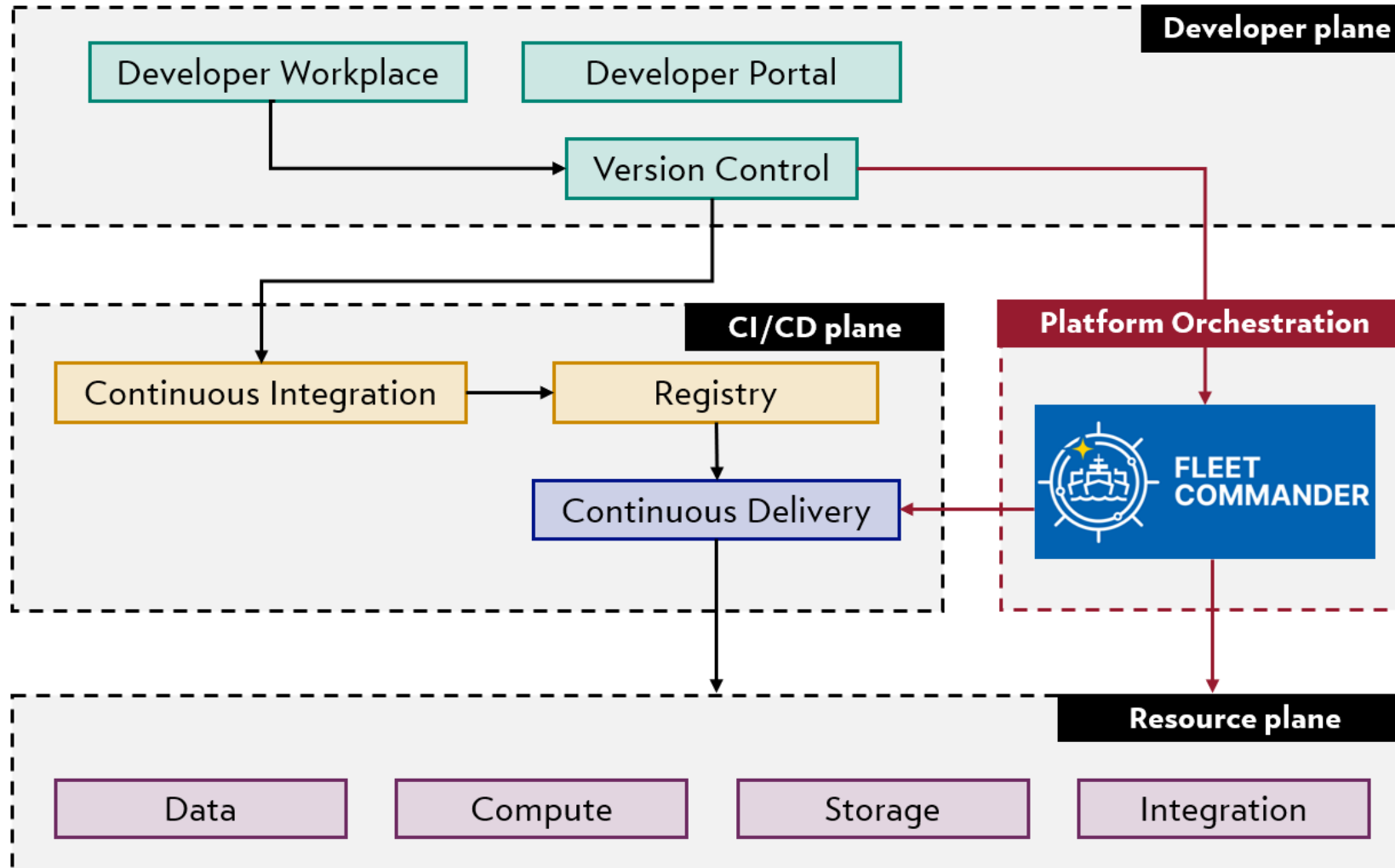
Provision a generic resource



Where our Journey concludes



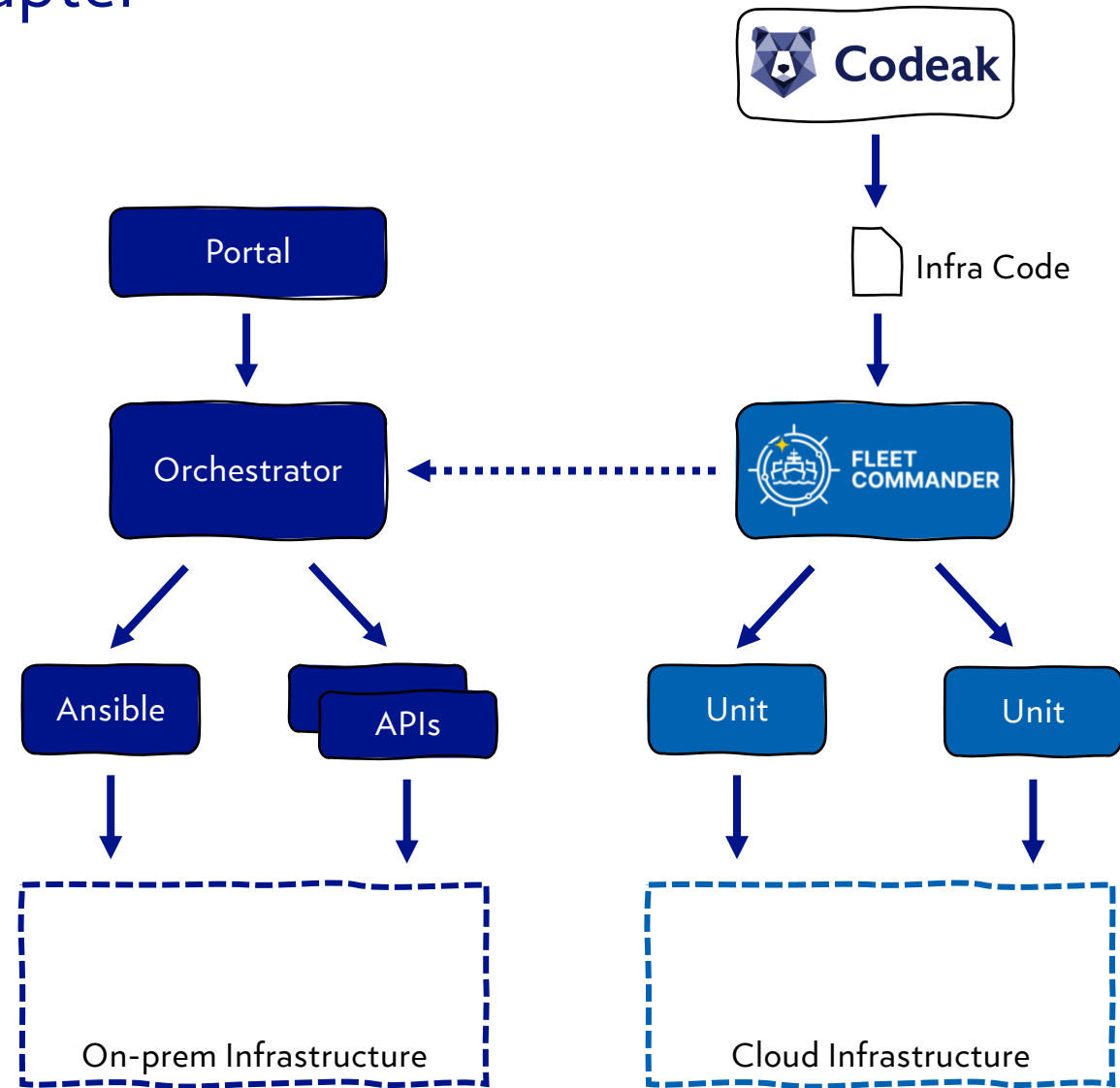
Codeak Internal DevOps Platform

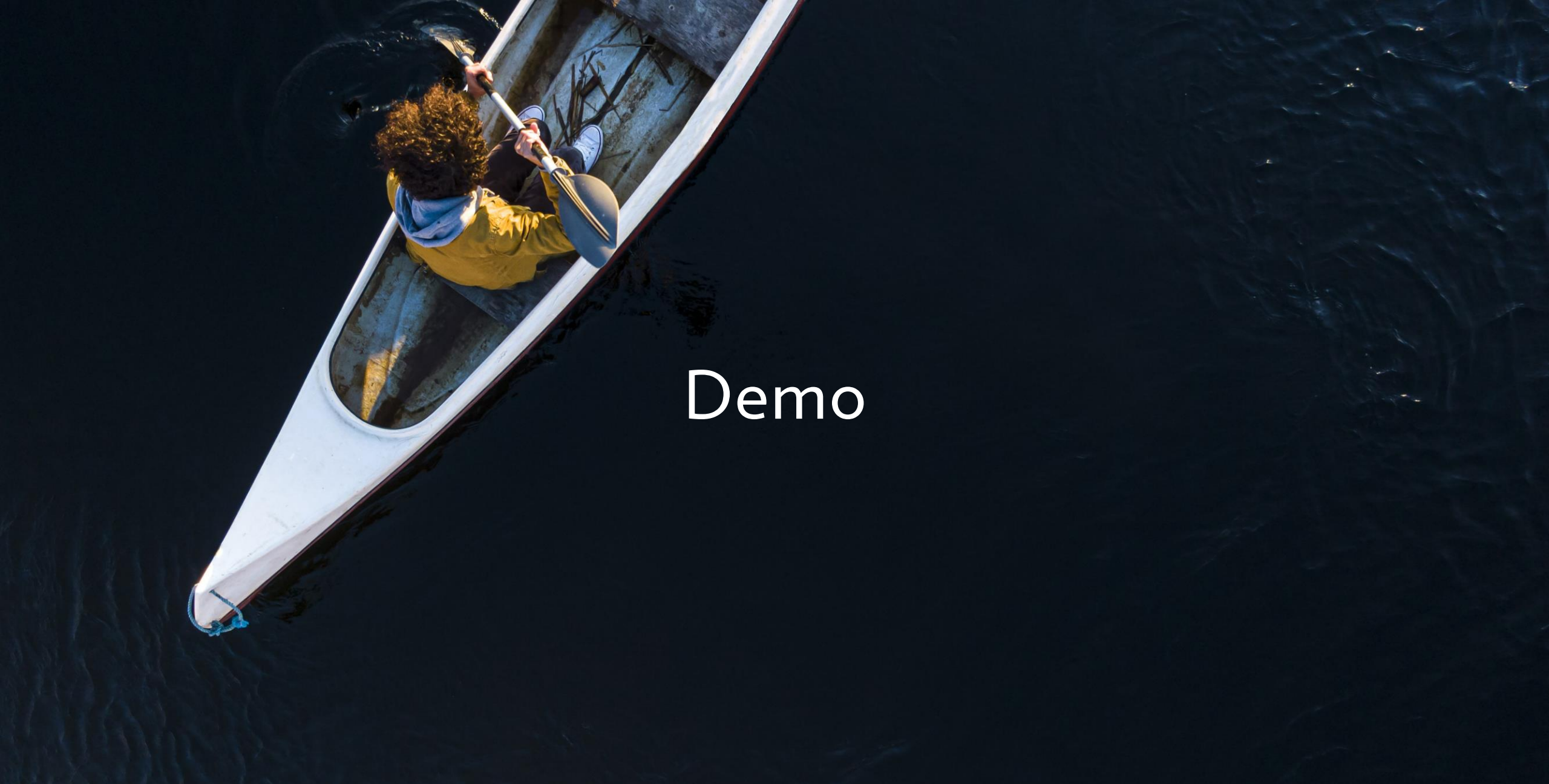


Our automation journey: Final chapter

Bringing it all together

- Automate each runtime with the best suited tech
- Classic workflow orchestration & control plane can co-exist (best of both worlds)
- **Gradual transition** from UIs to Infra as Code
- **No big-bang** migration necessary
- Strong integration of infrastructure ordering processes into our Internal Developer Platform
😊 making it a **DevOps** platform



A high-angle, top-down photograph of a person with curly hair, wearing a bright yellow jacket and a light blue hood, paddling a white canoe on dark, rippling water. The person is holding a double-bladed paddle. The canoe is white with a dark interior and a blue rope tied to the bow. The water is dark blue/black with small, light-colored ripples. The word "Demo" is written in large white letters to the right of the canoe.

Demo

.../github.com/BigGold1310/demo-distributed-cp

Cluster: kind-commander
No resources found

Cluster: kind-kubernetes
No resources found

Cluster: kind-storage
No resources found

Cluster: kind-iam
error: the server doesn't have a resource type "iamroles"

.../demo-distributed-cp [* kind-commander>] 🔍

🔗 main [📁]

Cluster: kind-commander
No resources found

.../github.com/BigGold1310/demo-distributed-cp

Cluster: kind-commander

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
commander	k8s-1	True	False	kubernetescluster	3s

Cluster: kind-kubernetes

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	False	kubernetescluster	2s

Cluster: kind-storage

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	False	storage	2s

Cluster: kind-iam

error: the server doesn't have a resource type "iamroles"

.../demo-distributed-cp [* kind-commander>] kc apply -f demo/clusters/cluster-1.yaml

kubernetescluster.public.demo.io/k8s-1 created

.../demo-distributed-cp [* kind-commander>]

main

main

Cluster: kind-commander

NAMESPACE	NAME	KIND	PROVIDERCONFIG	SYNCED	READY	AGE
commander	k8s-1-kubernetes	KubernetesCluster	kubernetes	True	False	2s
commander	k8s-1-storage	Storage	storage	True	False	2s

Cluster: kind-commander

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
commander	k8s-1	True	True	kubernetescluster	37s
commander	k8s-2	True	False	kubernetescluster	2s

Cluster: kind-kubernetes

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	kubernetescluster	37s
unit	k8s-2	True	False	kubernetescluster	2s

Cluster: kind-storage

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	storage	37s
unit	k8s-2	True	False	storage	2s

Cluster: kind-iam
error: the server doesn't have a resource type "iamroles"

```
.../demo-distributed-cp [* kind-commander>] kc apply -f demo/clusters/cluster-1.yaml
kubernetescluster.public.demo.io/k8s-1 created

.../demo-distributed-cp [* kind-commander>] kc apply -f demo/clusters/cluster-2.yaml
kubernetescluster.public.demo.io/k8s-2 created

.../demo-distributed-cp [* kind-commander>]
```

main []

main []

main []

Cluster: kind-commander

NAMESPACE	NAME	KIND	PROVIDERCONFIG	SYNCED	READY	AGE
commander	k8s-1-kubernetes	KubernetesCluster	kubernetes	True	True	37s
commander	k8s-1-storage	Storage	storage	True	True	37s
commander	k8s-2-kubernetes	KubernetesCluster	kubernetes	True	False	1s
commander	k8s-2-storage	Storage	storage	True	False	1s

Cluster: kind-commander

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
commander	k8s-1	True	True	kubernetescluster	2m47s
commander	k8s-2	True	True	kubernetescluster	2m10s

Cluster: kind-kubernetes

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	kubernetescluster	2m46s
unit	k8s-2	True	True	kubernetescluster	2m10s

Cluster: kind-storage

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	storage	2m46s
unit	k8s-2	True	True	storage	2m10s

Cluster: kind-iam

No resources found

```
.../demo-distributed-cp [* kind-commander>] kc apply -f demo/clusters/cluster-1.yaml
kubernetescluster.public.demo.io/k8s-1 created
```

```
.../demo-distributed-cp [* kind-commander>] kc apply -f demo/clusters/cluster-2.yaml
kubernetescluster.public.demo.io/k8s-2 created
```

```
.../demo-distributed-cp [* kind-commander>] kctx kind-iam
✓ Switched to context "kind-iam".
```

```
.../demo-distributed-cp [* kind-iam>] kc apply -f demo/units/unit-iam/composition.yaml
composition.apiextensions.crossplane.io/iamrole created
```

```
.../demo-distributed-cp [* kind-iam>] kc apply -f demo/units/unit-iam/xrd.yaml
compositeresourcedefinition.apiextensions.crossplane.io/iamroles.iam.unit.demo.io created
```

```
.../demo-distributed-cp [* kind-iam>] 
```

main []

main []

main []

main []

main []

main []

Cluster: kind-commander

NAMESPACE	NAME	KIND	PROVIDERCONFIG	SYNCED	READY	AGE
commander	k8s-1-kubernetes	KubernetesCluster	kubernetes	True	True	2m46s
commander	k8s-1-storage	Storage	storage	True	True	2m46s
commander	k8s-2-kubernetes	KubernetesCluster	kubernetes	True	True	2m10s
commander	k8s-2-storage	Storage	storage	True	True	2m10s

.../github.com/BigGold1310/demo-distributed-cp

Cluster: kind-commander

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
commander	k8s-1	True	False	kubernetescluster	5m3s
commander	k8s-2	True	False	kubernetescluster	4m20s

Cluster: kind-storage

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	storage	5m2s
unit	k8s-2	True	True	storage	4m20s

Cluster: kind-kubernetes

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	kubernetescluster	5m2s
unit	k8s-2	True	True	kubernetescluster	4m20s

Cluster: kind-iam

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
default	k8s-1-iam	True	False	iamrole	20s
default	k8s-2-iam	True	False	iamrole	20s

.../demo-distributed-cp [* kind-iam>] kctx kind-commander

✓ Switched to context "kind-commander".

.../demo-distributed-cp [* kind-commander>] kc apply -f demo/commander/composition-v2.yaml

composition.apiextensions.crossplane.io/kubernetescluster configured

.../demo-distributed-cp [* kind-commander>] |

main

main

main

Cluster: kind-commander

NAMESPACE	NAME	KIND	PROVIDERCONFIG	SYNCED	READY	AGE
commander	k8s-1-iam	IAMRole	iam	True	False	20s
commander	k8s-1-kubernetes	KubernetesCluster	kubernetes	True	True	5m2s
commander	k8s-1-storage	Storage	storage	True	True	5m2s
commander	k8s-2-iam	IAMRole	iam	True	False	20s
commander	k8s-2-kubernetes	KubernetesCluster	kubernetes	True	True	4m20s
commander	k8s-2-storage	Storage	storage	True	True	4m20s

.../github.com/BigGold1310/demo-distributed-cp

Cluster: kind-commander

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
commander	k8s-1	True	False	kubernetescluster	5m37s
commander	k8s-2	True	False	kubernetescluster	5m
commander	k8s-3	True	False	kubernetescluster	1s

Cluster: kind-storage

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	storage	5m34s
unit	k8s-2	True	True	storage	5m
unit	k8s-3	True	False	storage	1s

Cluster: kind-kubernetes

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
unit	k8s-1	True	True	kubernetescluster	5m34s
unit	k8s-2	True	True	kubernetescluster	5m
unit	k8s-3	True	False	kubernetescluster	1s

Cluster: kind-iam

NAMESPACE	NAME	SYNCED	READY	COMPOSITION	AGE
default	k8s-1-iam	True	True	iamrole	55s
default	k8s-2-iam	True	True	iamrole	55s
default	k8s-3-iam	True	False	iamrole	1s

.../demo-distributed-cp [* kind-iam>] kctx kind-commander
✓ Switched to context "kind-commander".

.../demo-distributed-cp [* kind-commander>] kc apply -f demo/commander/composition-v2.yaml
composition.apiextensions.crossplane.io/kubernetescluster configured

.../demo-distributed-cp [* kind-commander>] kc apply -f demo/clusters/cluster-3.yaml
kubernetescluster.public.demo.io/k8s-3 created

.../demo-distributed-cp [* kind-commander>]

main

KIND	PROVIDERCONFIG	SYNCED	READY	AGE
IAMRole	iam	True	False	54s
KubernetesCluster	kubernetes	True	True	5m35s
Storage	storage	True	True	5m35s
IAMRole	iam	True	False	54s
KubernetesCluster	kubernetes	True	True	4m59s
Storage	storage	True	True	4m59s
IAMRole	iam	True	False	0s
KubernetesCluster	kubernetes	True	False	0s
Storage	storage	True	False	0s

Who are we?



Cyrill Näf

Platform Engineer

in linkedin.com/in/cyrill-naef



Angelo Gargiulo

Product Owner IT Automation

in linkedin.com/in/gargiuloangelo